Code Walkthrough & Run Guide

Classification of Human-Written and AI-Generated Text
National Institute of Technology Sikkim — CSE

Understand the workflow end-to-end • How to train, test, and serve the model in 6 ways
Dual TF-IDF (75k features) • Logistic Regression & Linear SVM — 99%+ test accuracy

Authors: **Pravesh Gedon Sharma (B240019)** & **Ankit Rawat (B320032)** • This PDF pairs with `Code_Workflow_and_Run_Guide.pdf` (this file) and `Project_Report_AI_Text_Classification.pdf` (results).

Contents

1 Overview & Architecture

2 Repository Map (file by file)

3 src/ — Shared Utilities

4 Training Workflow (train_baseline.py, 11 steps)

5 Inference Workflow (test/inference.py)

6 The 6 Ways to Run It (CLI, Folder, CSV, Terminal, API, LangChain, Streamlit)

7 How to Run — Copy-Paste Commands

8 Outputs & What Each File Means

9 Troubleshooting & FAQ

Appendix A: Key Snippets (load → vectorize → predict)

1 Overview & Architecture

The project turns an essay into a 75,000-dimensional sparse vector (25k word TF-IDF + 50k char TF-IDF) and asks a linear classifier: **Human or AI?**. Nothing else. That simplicity is the point — it trains in 33 seconds, runs on a laptop, exposes a REST API, a LangChain tool, a CLI, and a web UI, all from the same 4 `.pkl` files.

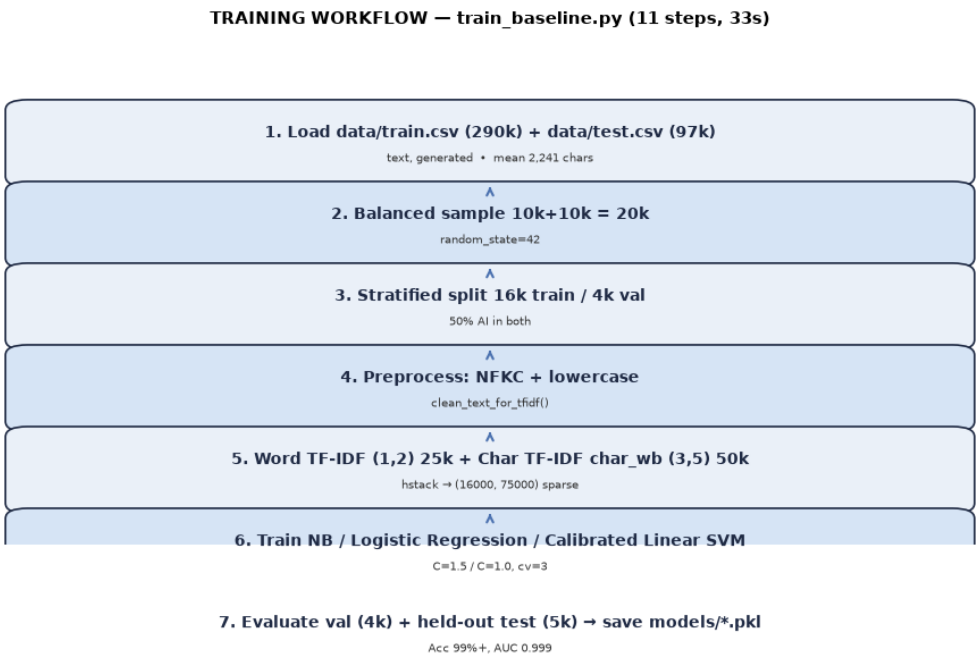


Figure 1: Training flow.

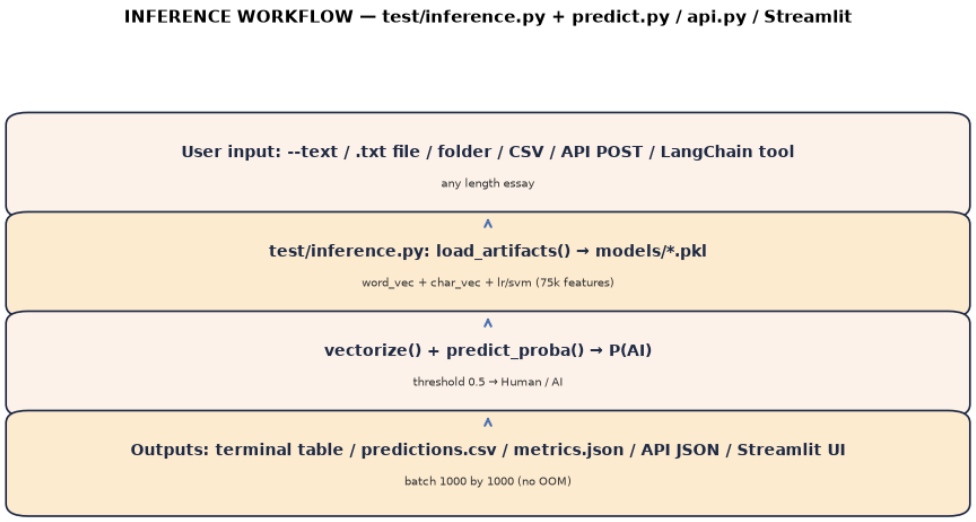


Figure 2: Inference flow (all 6 front-ends share test/inference.py).

- **Training (once):** `train_baseline.py` → `models/*.pkl` + `results/baseline_ml_metrics.csv`
- **Inference (every time):** `test/inference.py` loads the 4 pkls → `predict.py` / `api.py` / `langchain_tool.py` / `app/app.py` (Streamlit) → terminal answer + optional CSV/JSON.
- **Data:** `data/train.csv` 290,300 rows (balanced) / `data/test.csv` 97,447 rows. Columns: `text, generated` (0=Human, 1=AI).

2 Repository Map (file by file)

File	What it does	Lines
`train_baseline.py`	Full training pipeline (11 logged steps). Load → sample → split → TF-IDF → train NB/LR/SVM → evaluate → save.	142
`make_report.py`	Generates Project_Report_*.pdf (results + 5 figures) from live artifacts.	319
`make_run_guide.py`	Generates THIS guide PDF.	this
`src/preprocessing.py`	clean_text_for_tfidf() — NFKC + lowercase for TF-IDF.	45
`src/stylometrics.py`	TTR, burstiness, Flesch ease/grade, punctuation rates — used by Streamlit + tests.	162
`src/evaluate.py`	compute_classification_metrics(), plot_confusion_matrix(), plot_roc_curves().	95
`src/semantic.py`	Sentence-BERT embeddings + cosine — Phase 2 (not yet wired into inference).	80
`app/app.py`	Streamlit web UI (:8501). Loads models/ with @cache_resource, shows P(AI) + stylometrics.	110
`test/inference.py`	**Core inference engine.** load_artifacts(), vectorize(), predict_text/batch, evaluate_csv(), predict_folder().	158
`test/predict.py`	CLI: --text / --input / --folder / --csv / --interactive. Args → inference.py.	140
`test/api.py`	FastAPI (:8001): POST /predict, /predict/batch, /langchain/tool, GET /health, /model/info.	95
`test/langchain_tool.py`	LangChain BaseTool ai_human_classifier + batch tool. Agent-ready.	86
`test/terminal_demo.py`	Terminal demo comparing direct vs langchain vs API in one run (+ --loop chat).	81
`test/evaluate_examples.py`	Reads all test/example_texts/*.txt, predicts both models, prints aggregate metrics.	62
`test/test_model.py`	Smoke tests: single predict + 2k eval (assert acc>0.90). --full = 97k.	33

3 src/ — Shared Utilities

preprocessing.py

`clean_text_for_tfidf(text)` = NFKC → collapse whitespace → lowercase. Used as `preprocessor=` in word TF-IDF so train and test are normalised identically. `clean_text_for_stylometrics()` keeps casing/punctuation (capital ratio, commas) — never lowercases.

stylometrics.py

`extract_stylometric_features(text) → 24 floats` (num_words, ttr, hapax_ratio, mean_sentence_length, burstiness=σ/μ, comma_rate… flesch_reading_ease, flesch_kincaid_grade, gunning_fog). No sklearn — pure regex + syllable heuristics. Shown in Streamlit and returned by `--stylometrics` / API `stylometrics:true`.

evaluate.py / semantic.py

`evaluate.py` provides notebook plots (confusion + ROC) and metric dict. `semantic.py` (Sentence-BERT) is Phase-2 scaffolding — researched but **not** part of the 75k TF-IDF inference path.

4 Training Workflow (train_baseline.py, 11 steps, 33 s)

Run: `venv/bin/python train_baseline.py`. Each step prints a timestamped log line — watch them in order:

Step	What happens	Output
1	Load data/train.csv (290,300) + data/test.csv (97k metadata). Prints shape, class counts, length stats.	train (290300,2)
2	Balanced sample: per-class `sample(10000)` → concat → shuffle. Why 20k? Full 290k×75k TF-IDF is 14× slower; notebook 03 defines 20k.	(20000,2) 50/50
3	Stratified split (random_state=42, test_size=0.20) → 16k train / 4k val, both 50% AI.	train=16000 val=4000
4	Preprocessing demo: prints raw vs cleaned (lowercase) for 2 rows.	clean_text_for_tfidf
5	Word TF-IDF (1,2) 25k, min_df=2, sublinear_tf. fit on train, transform val.	(16000,25000) in 6.2s
6	Char TF-IDF char_wb (3,5) 50k, min_df=3. fit/transform.	(16000,50000) in 15.8s
7	Fusion hstack(word,char).tocsr(). Prints nnz (non-zero cells).	(16000,75000) nnz 34.8M
8	Train NB / LR (C=1.5) / LinearSVC+CalibratedCV(3). Each evals on 4k val: accuracy, precision, recall, F1, ROC-AUC, confusion, full report.	SVM 99.53% acc
9	Save results/baseline_ml_metrics.csv (Model,accuracy,…).	results/*.csv
10	Held-out test: load test.csv, sample 5000 (seed 42), transform with frozen vectorizers, LR+SVM eval.	LR 99.26% SVM 99.58%
11	Save 4 pkls to models/. Prints TOTAL TIME 33s.	models/*.pkl

Two pandas-3 fixes are baked in (your run hit them): per-class sampling instead of `groupby.apply` (which drops `generated`), and `np.array(..., dtype=object)` for Arrow-backed strings before sklearn split.

5 Inference Workflow (test/inference.py)

`test/inference.py` is the **one place** that touches models. Every front-end (CLI, API, LangChain, Streamlit) calls it — change inference once, all 6 ways update.

Function	One-liner
`load_artifacts()`	joblib.load ×4 from ../models/. Raises FileNotFoundError with 'Train first' hint if missing.
`vectorize(texts, art)`	word.transform + char.transform → hstack().tocsr(). Single str auto-wrapped to list.
`predict_proba(texts, art, model)`	clf.predict_proba(X)[:,1] → P(AI) array. model='lr'|'svm'.
`predict_text(text, art, model, stylometrics)`	Single-text dict: predicted_label (≥0.5→AI), probability_ai/human, confidence, optional 24 stylometrics.
`predict_batch(texts, batch_size=1000)`	Chunked to avoid OOM on 97k/290k CSVs. Returns np.array of P(AI).

Function	One-liner
`evaluate_csv(csv, sample_n)`	Loads CSV, batches predict, computes accuracy/precision/recall/F1/AUC + confusion + report, returns (metrics, df with prob_ai/pred/correct).
`predict_folder(folder)`	glib *.txt/*.md, predict_text each + stylometrics + file path.

6 The 6 Ways to Run It

#	Way	Command / Endpoint	Best for
1	CLI single / file	`test/predict.py --text "..." / --input file.txt`	One-off classify
2	CLI folder / CSV	`--folder test/example_texts` / `--csv data/test.csv --sample 5000`	Batch eval
3	Terminal chat	`test/terminal_demo.py --loop` or `predict.py --interactive`	Try text live
4	REST API	`test/api.py` → POST `/predict` (8001)	Other programs / curl
5	LangChain tool	`langchain_tool.ai_human_classifier.run(...)`	LLM agents
6	Web UI	`app/app.py` (Streamlit :8501)	Demo to humans

7 How to Run — Copy-Paste Commands (from project root)

A. First-time setup (once)

```
cd ~/Desktop/AI_generated_text_classification
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install "langchain-core>=0.3" "fastapi>=0.110" reportlab
# data: ensure data/train.csv (623MB) + data/test.csv (212MB) exist
ls -lh data/
```

B. Train (once — 33 seconds with data present)

```
venv/bin/python train_baseline.py
# → models/word_tfidf_vectorizer.pkl (969K)
#   models/char_tfidf_vectorizer.pkl (1.6M)
#   models/logistic_regression_baseline.pkl (587K)
#   models/linear_svm_baseline.pkl (1.7M)
#   results/baseline_ml_metrics.csv
```

C. Test / Inference — all 6 ways

```
# 1 — single text (terminal)
venv/bin/python test/predict.py --text "Your essay here..." --model lr --stylometrics

# 2 — file / folder (13 examples in test/example_texts/)
venv/bin/python test/predict.py --input test/example_texts/human_essay_school.txt --model svm
venv/bin/python test/predict.py --folder test/example_texts --model lr
venv/bin/python test/evaluate_examples.py          # custom evaluator (both models, aggregate)

# 3 — terminal chat (no browser)
venv/bin/python test/terminal_demo.py
venv/bin/python test/terminal_demo.py --api        # needs API running (see D)
venv/bin/python test/predict.py --interactive --model lr

# 4 — REST API (server, second terminal)
venv/bin/python test/api.py                        # → http://localhost:8001/docs
curl -X POST http://localhost:8001/predict -H "Content-Type: application/json" \
  -d '{"text":"In todays rapidly evolving digital landscape...","model":"lr","stylometrics":true}'

# 5 — LangChain tool (inside Python / agent)
venv/bin/python test/langchain_tool.py             # no-LLM demo
#   in agent: from test.langchain_tool import ai_human_classifier

# 6 — Web UI (second terminal)
venv/bin/streamlit run app/app.py --server.port 8501   # → http://localhost:8501
```

D. Full evaluation on whole files (batch inference)

```
venv/bin/python test/predict.py --csv data/test.csv --model lr --out test/results          # 97k rows
venv/bin/python test/predict.py --csv data/train.csv --model svm --out test/results_train_full # 290k rows
venv/bin/python test/test_model.py             # 2k-row smoke test
venv/bin/python test/test_model.py --full      # full 97k
```

E. Report PDF

```
venv/bin/python make_report.py        # → Project_Report_AI_Text_Classification.pdf (361KB)
venv/bin/python make_run_guide.py     # → Code_Workflow_and_Run_Guide.pdf (this file)
```

8 Outputs & What Each File Means

File	What it is
`models/*.pkl (4)`	Frozen vectorizers + classifiers. The ONLY things inference loads. Retrain to replace.
`results/baseline_ml_metrics.csv`	Val metrics per model (train_baseline.py step 9).
`test/results/predictions.csv`	Per-row: text, generated, prob_ai, pred, correct. Review errors here.
`test/results/metrics.json`	Same metrics as terminal + confusion matrix + full classification report.
`test/example_texts/`	13 .txt examples (human_/ai_ prefix = ground truth). `evaluate_examples.py` scores them.
`test/example_test_set.csv`	20-row balanced CSV for quick batch eval (--csv test/example_test_set.csv).
`results/figures/*.png`	5 figures for the PDF (class dist, length, comparison, confusion, ROC).

9 Troubleshooting & FAQ

- **FileNotFoundError: models/*.pkl missing** → you haven't trained yet. Run `venv/bin/python train_baseline.py` (needs `data/train.csv`).
- **KeyError 'generated' / Arrow TypeError** on pandas 3 → already fixed in train_baseline.py (per-class sampling + np.array dtype=object). Do not revert to groupby.apply.
- **API 'Connection refused' on 8001** → API not running. Start `venv/bin/python test/api.py` in a **separate terminal** (keep it open).
- **uvicorn test.api:app fails** → `test/` needs `__init__.py` (added) OR run `python test/api.py` directly (recommended).
- **Short text predicted wrong** (e.g. human_sample.txt 26 words → AI) → expected. Model is trained on ~400-word essays (mean 2,241 chars). Very short/clean text is out-of-distribution. Test with essay-length paths.
- **Streamlit not showing new model** → kill old process (`pkill -f "streamlit run"`) then restart — @cache_resource caches artifacts.
- **OOM on full train.csv** → don't load whole CSV in memory; use `--csv ... --sample 5000` or batch_size=1000 in `evaluate_csv()`.

Appendix A Key Snippets (load → vectorize → predict)

This is the entire inference in 10 lines — everything else is wrapper around it:

```
import joblib
from scipy.sparse import hstack

word_vec = joblib.load("models/word_tfidf_vectorizer.pkl")
char_vec = joblib.load("models/char_tfidf_vectorizer.pkl")
lr       = joblib.load("models/logistic_regression_baseline.pkl")

text = open("test/example_texts/ai_essay_distance_learning.txt").read()
X = hstack([word_vec.transform([text]), char_vec.transform([text])]).tocsr()
p_ai = lr.predict_proba(X)[0, 1]
print("AI-Generated" if p_ai >= 0.5 else "Human-Written", f"P(AI)={p_ai:.4f}")
# → AI-Generated P(AI)=0.9856
```

Higher-level: `from test.inference import load_artifacts, predict_text; r = predict_text(text, load_artifacts(), model='lr')`; REST: `POST /predict`; LangChain: `ai_human_classifier.run({'text': text})`.

Generated automatically by `make_run_guide.py` *from live artifacts.*